Everything Python Programming by Joel Lee

This is a comprehensive cheatsheet for Python programming prepared for reference purposes.

Variables and Data Types

Features for declaring and managing variables and data types in Python.

In Python, data types can be broadly categorized into primitive types (immutable built-in types like numbers, strings, and booleans) and reference types (mutable or immutable collections that are typically referenced and can be modified or structured in complex ways). Note that in Python, all variables are references to objects, but this distinction helps in understanding mutability and usage patterns.

The `type` function is a built-in utility for checking the type of any object and is listed separately.

Primitive Data Types

Features: int, float, str, bool, None

Feature	Uses	Example	Practical Uses	Common Methods
int	Integer data type.	`x = 42`	Counters, IDs, or loop indices.	`bit_length()`, `to_bytes()`, `from_bytes()`
float	Floating-point number.	`price = 19.99`	Prices, measurements, or calculations.	`is_integer()`, `as_integer_ratio()`, `hex()`
str	String data type.	`name = "Python"`	Text for user interfaces or logs.	`upper()`, `lower()`, `strip()`, `replace()`, `split()`, `join()`, `format()`
bool	Boolean (True/False) type.	`is_active = True`	Conditional logic for toggles.	None (inherits from int, e.g., `__and__`, `__or__`)
None	Represents absence of value.	`result = None`	Default or uninitialized states.	None

Reference Data Types (Collections)

Features: list, tuple, dict, set

Feature	Uses	Example	Practical Uses	Common Methods
list	Mutable ordered collection.	`items = [1, 2, 3]`	Storing dynamic lists	`append()`, `pop()`, `remove()`, `extend()`,

Feature	Uses	Example	Practical Uses	Common Methods
			like user inputs.	`insert()`, `clear()`, `sort()`
tuple	Immutable ordered collection.	`coords = (10, 20)`	Fixed data like coordinates or constants.	`count()`, `index()`
dict	Key-value pair collection.	`user = {"name": "John", "age": 30}`	Storing structured data like JSON.	`get()`, `keys()`, `values()`, `items()`, `pop()`, `update()`, `clear()`
set	Unordered collection of unique items.	`tags = {"new", "sale"}`	Removing duplicates or set operations.	`add()`, `remove()`, `discard()`, `union()`, `intersection()`, `difference()`

Type Checking

Feature	Uses	Example	Practical Uses	Common Methods
type	Returns type of an object.	`type(42)`	Debugging or type checking.	None

Operators

Features for performing operations on variables and values.

Features: arithmetic, comparison, logical, bitwise, assignment, membership, identity

Feature	Uses	Example	Practical Uses
Arithmetic	Mathematical operations.	`x = 5 + 3; y = 10 / 2`	Calculations like totals or averages.
Comparison	Compares values.	`if x > y: print("Greater")`	Sorting or filtering data.
Logical	Combines boolean expressions.	`if x > 0 and y < 10: print("Valid")`	Complex condition checks.
Bitwise	Operates on binary representations.	`z = x & y`	Low-level programming or flags.
Assignment	Assigns values to variables.	`x += 1`	Updating counters or accumulators.
Membership	Checks if value is in a collection.	`if "a" in ["a", "b"]: print("Found")`	Validating list or string contents.
Identity	Checks object identity.	`if x is None: print("None")`	Checking for None or object equality.

Operator Details

- **Arithmetic**: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `//` (floor division), `%` (modulus), `**` (exponentiation)
- **Comparison**: `==` (equal), `!=` (not equal), `<` (less than), `>` (greater than), `<=` (less than or equal), `>=` (greater than or equal)
- **Logical**: `and`, `or`, `not`
- **Bitwise**: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (left shift), `>>` (right shift)
- **Assignment**: `=`, `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, `&=`, `|=`, `^=`, `<<=`, `>>=`
- **Membership**: `in`, `not in`
- **Identity**: `is`, `is not`

Control Flow

Features for controlling program execution.

Features: if, elif, else, for, while, break, continue, pass, try-except

Feature	Uses	Example	Practical Uses
if	Conditional execution.	`if x > 0: print("Positive")`	Validating user input.
elif	Additional conditional check.	`if x > 0: pass elif x == 0: print("Zero")`	Multi-level conditions like grading.
else	Alternative execution path.	`if x: pass else: print("False")`	Handling default cases.
for	Iterates over sequences.	`for i in range(5): print(i)`	Looping through lists or files.
while	Loops while condition is true.	`while x > 0: x -= 1`	Polling or countdowns.
break	Exits loop.	`for i in range(5): if i == 3: break`	Early loop termination.
continue	Skips current loop iteration.	`for i in range(5): if i == 2: continue`	Skipping invalid data.
pass	No-op placeholder.	`def func(): pass`	Stub functions or empty blocks.
try-except	Handles exceptions.	`try: x = 1/0 except ZeroDivisionError: print("Error")`	Handling file or network errors.

Functions

Features for defining and using reusable code blocks.

Features: def, return, lambda, *args, **kwargs, global, nonlocal

Feature	Uses	Example	Practical Uses
def	Defines a function.	`def add(a, b): return a + b`	Reusable logic like calculations.
return	Exits function with value.	`def get_name(): return "John"`	Returning data to callers.
lambda	Creates anonymous function.	`double = lambda x: x * 2`	Inline functions for map or filter.
*args	Accepts variable positional arguments.	`def sum_nums(*args): return sum(args)`	Flexible function inputs.
**kwargs	Accepts variable keyword arguments.	`def print_info(**kwargs): print(kwargs)`	Handling optional parameters.
global	Accesses global variable.	`global x; x = 10`	Modifying global state (use sparingly).
nonlocal	Accesses outer scope variable.	`def outer(): x = 1; def inner(): nonlocal x; x = 2`	Nested function state management.

Lists and Collections

Features for working with collections (building on reference data types).

Features: append, pop, remove, extend, list comprehension, slice, set.add, dict.get

Feature	Uses	Example	Practical Uses
append	Adds item to list end.	`items.append(4)`	Adding items to a dynamic list.
pop	Removes and returns last item.	`items.pop()`	Removing items from a stack.
remove	Removes first matching item.	`items.remove(2)`	Deleting specific list items.
extend	Adds multiple items to list.	`items.extend([5, 6])`	Merging lists like batch updates.
list comprehension	Creates list from expression.	`[x * 2 for x in range(5)]`	Generating transformed lists.
slice	Extracts portion of sequence.	`items[1:3]`	Paginating or subsetting data.

Feature	Uses	Example	Practical Uses
set.add	Adds item to set.	`tags.add("new")`	Adding unique tags to filters.
dict.get	Retrieves value with default.	`user.get("name", "Unknown")`	Safe access to dictionary keys.

String Manipulation

Built-in string methods and operations (building on the primitive str type).

Features: len, upper, lower, strip, replace, split, join, format

Feature	Uses	Example	Practical Uses
len	Returns string length.	`len("Hello")`	Validating input length.
upper	Converts to uppercase.	`"hello".upper()`	Formatting display text.
lower	Converts to lowercase.	`"HELLO".lower()`	Normalizing search queries.
strip	Removes leading/trailing whitespace.	`"  hi  ".strip()`	Cleaning user input.
replace	Replaces substring.	`"hello".replace("h", "H")`	Updating text in templates.
split	Splits string into list.	`"a,b,c".split(",")`	Parsing CSV data.
join	Joins list into string.	`",".join(["a", "b"])`	Creating formatted output.
format	Formats string with placeholders.	`"Hi, {}".format("John")`	Dynamic text in messages.

Object-Oriented Programming

Features for defining and working with classes and objects.

Features: class, **init**, self, inheritance, @staticmethod, @classmethod, property

Feature	Uses	Example	Practical Uses
class	Defines a blueprint for objects.	`class User: pass`	Modeling entities like users.
init	Initializes object instances.	`class User: def __init__(self, name): self.name = name`	Setting up object state.
self	Refers to current instance.	`self.name = name`	Accessing instance attributes.

Feature	Uses	Example	Practical Uses
inheritance	Extends a parent class.	`class Admin(User): pass`	Reusing code for user types.
@staticmethod	Defines method without self.	`@staticmethod def validate(): return True`	Utility methods in classes.
@classmethod	Defines method with class as first argument.	`@classmethod def create(cls): return cls()`	Alternative constructors.
property	Defines getter/setter methods.	`@property def name(self): return self._name`	Controlled attribute access.

File Handling

Features for reading and writing files.

Features: open, read, write, close, with, readline

Feature	Uses	Example	Practical Uses
open	Opens a file for reading/writing.	`f = open("data.txt", "r")`	Accessing log or config files.
read	Reads entire file content.	`content = f.read()`	Loading file data for processing.
write	Writes string to file.	`f.write("Hello")`	Saving user data to files.
close	Closes file handle.	`f.close()`	Releasing file resources.
with	Manages file context automatically.	`with open("data.txt") as f: content = f.read()`	Safe file handling.
readline	Reads one line from file.	`line = f.readline()`	Processing large files line-by-line.

Modules and Standard Library

Common Python modules and functions for extended functionality.

Features: import, math, random, datetime, os, sys, json

Feature	Uses	Example	Practical Uses
import	Imports modules or functions.	`import math`	Accessing standard library functions.
math	Mathematical functions and constants.	`math.sqrt(16)`	Calculations in data analysis.
random	Generates random numbers.	`random.randint(1, 10)`	Randomizing quiz questions.
datetime	Handles dates and times.	`from datetime import datetime; now = datetime.now()`	Timestamping logs or events.
os	Interacts with operating system.	`os.path.exists("file.txt")`	Checking file existence.
sys	System-specific parameters and functions.	`sys.exit(1)`	Terminating scripts on error.
json	Handles JSON encoding/decoding.	`json.loads('{"name": "John"}')`	Parsing API responses.

Exception Handling

Features for managing errors and exceptions (expanding on try-except from Control Flow).

Features: try, except, else, finally, raise

Feature	Uses	Example	Practical Uses
try	Monitors block for exceptions.	`try: x = 1/0`	Wrapping risky operations.
except	Handles specific exceptions.	`except ZeroDivisionError: print("Error")`	Catching division errors.
else	Runs if no exception occurs.	`try: x = 1 except: pass else: print("Success")`	Post-success logic.
finally	Runs regardless of exception.	`finally: print("Done")`	Closing resources like files.
raise	Throws an exception.	`raise ValueError("Invalid input")`	Custom error handling.

Functional Programming

Features for functional-style programming.

Features: map, filter, reduce, lambda (already covered), list comprehension (already covered)

Feature	Uses	Example	Practical Uses
map	Applies function to each item in iterable.	`list(map(lambda x: x * 2, [1, 2, 3]))`	Transforming data for display.
filter	Filters items by condition.	`list(filter(lambda x: x > 0, [-1, 0, 1]))`	Filtering valid records.
reduce	Combines items to single value.	`from functools import reduce; reduce(lambda x, y: x + y, [1, 2, 3])`	Summing values in lists.